

Projekt 6: Problem wieloagentowy środowisko CollectCoins

Szymon Kowalski

Michał Król

17 czerwca 2026

1 Opis środowiska

Zaimplementowaliśmy własne środowisko wieloagentowe **CollectCoins** w API PettingZoo (**ParallelEnv**). Dwóch agentów (**agent_0**, **agent_1**) porusza się po dyskretnej siatce 9×9 . W każdym epizodzie pojawiają się cztery monety do zebrania w limicie 65 kroków. Akcje: *stay*, *up*, *down*, *left*, *right*.

Mapa zawiera stałe przeszkody (dwa słupy ścian), a obserwacja agenta obejmuje: własną pozycję, pozycję partnera, współrzędne monet, ułamek czasu epizodu oraz cztery sensory ścian (czy ruch w danym kierunku jest możliwy).

Nagroda: +1 za zebraną monetę (wspólna), niewielki *shaping* za zbliżenie się do monety oraz kara $-0,01$ za każdy krok.

2 Zastosowane algorytmy

Wykorzystaliśmy bibliotekę **Stable-Baselines3** (trening na CPU, polityka MLP 128×128):

- **PPO** — wariant **v0** ($\text{lr} = 3 \cdot 10^{-4}$) oraz **v1** ($\text{lr} = 10^{-3}$, większa entropia),
- **A2C** — wariant **v0**.

3 Eksperymenty

3.1 Ten sam algorytm (parameter sharing)

Jeden model trenujemy na pojedynczym agencie (partner losowy), a w ewaluacji **ten sam model** steruje oboma agentami na podstawie ich własnych obserwacji.

3.2 Różne algorytmy w epizodzie

agent_0 sterowany jest przez PPO, **agent_1** przez A2C; oba modele grają wspólnie w jednym epizodzie.

Dla każdej konfiguracji wykonaliśmy 3 niezależne uruchomienia (seedy). Budżet treningu: 200 000 kroków (shared) lub $2 \times 100\,000$ (mixed). Ewaluacja co 5 000 kroków na 20 epizodach testowych.

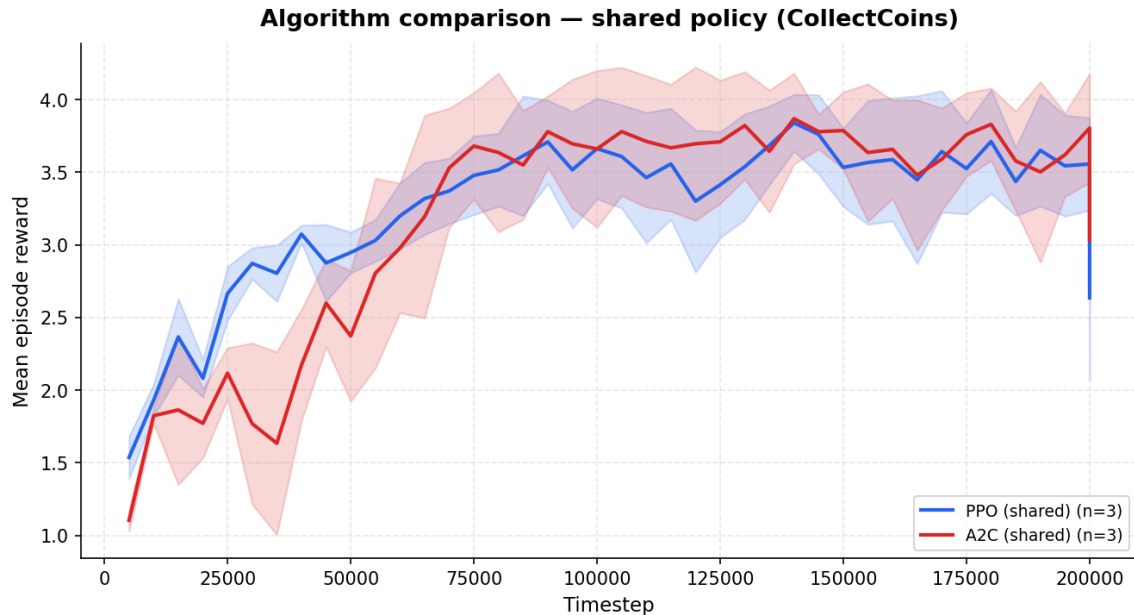
4 Wyniki i krzywe uczenia

Tabela 1 zestawia średnią nagrodę zespołu na początku i na końcu treningu oraz wartość maksymalną na krzywej uczenia (uśrednione po seedach).

Na wykresach widać wyraźny wzrost nagrody w trakcie uczenia. Najwyższe wyniki osiąga konfiguracja mieszana (PPO + A2C), osiągając ok. 4.0 nagrody/epizod (przy maksymalnie ok. 4 monetach plus shaping).

Tabela 1: Podsumowanie krzywych uczenia (średnia nagroda zespołu).

Konfiguracja	Początek	Koniec	Szczyt
PPO v0 (shared)	+1.54	+2.64	+3.84
PPO v1 (shared)	+1.65	+2.74	+3.69
A2C (shared)	+1.10	+3.03	+3.87
Mixed PPO+A2C	+2.73	+4.08	+4.08



Rysunek 1: Porównanie algorytmów PPO i A2C (ten sam algorytm, parameter sharing).

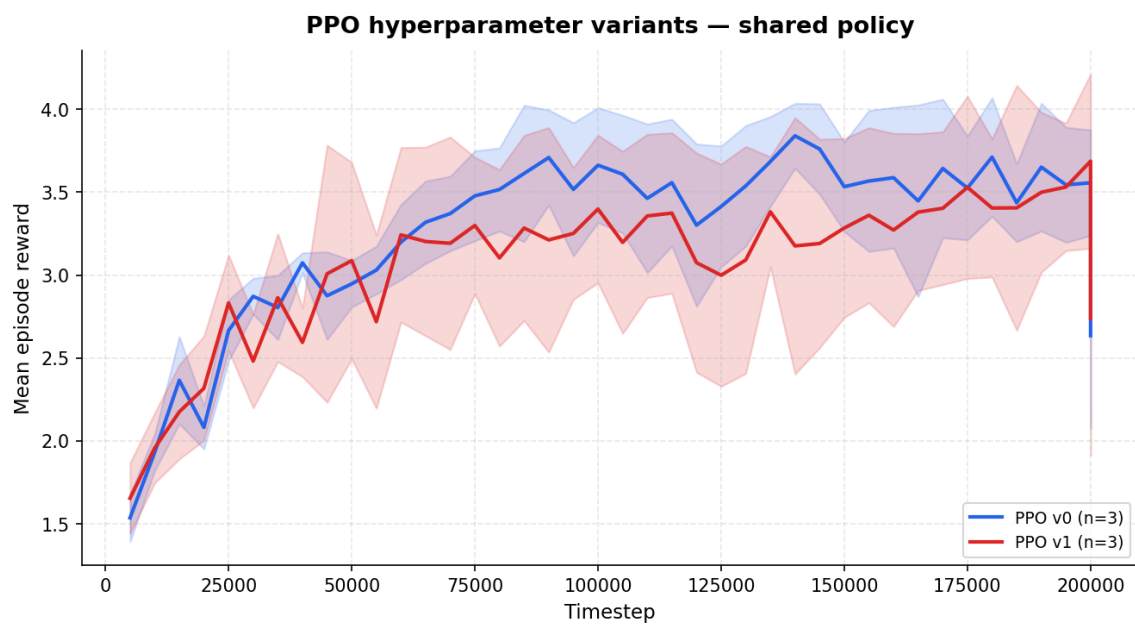
5 Wnioski

- Agenci uczą się zbierać monety we współpracy — krzywe uczenia rosną z ok. 1–1.5 do ok. 3–4 nagrody zespołu.
- **A2C** osiąga porównywalne wyniki do PPO w trybie shared, przy nieco większej zmienności.
- Wariant PPO z wyższym learning rate (v1) uczy się szybciej na początku, lecz końcowa jakość jest zbliżona do wariantu domyślnego.
- Konfiguracja **mixed** (różne algorytmy) osiąga najlepsze wyniki końcowe, co sugeruje, że specjalizacja agentów (PPO + A2C) może pomóc w koordynacji.
- Stała mapa ze ścianami i sensory w obserwacji są istotne dla stabilnego uczenia — wcześniejsze próby z losowymi ścianami bez pełnej obserwacji dawały słabe wyniki.

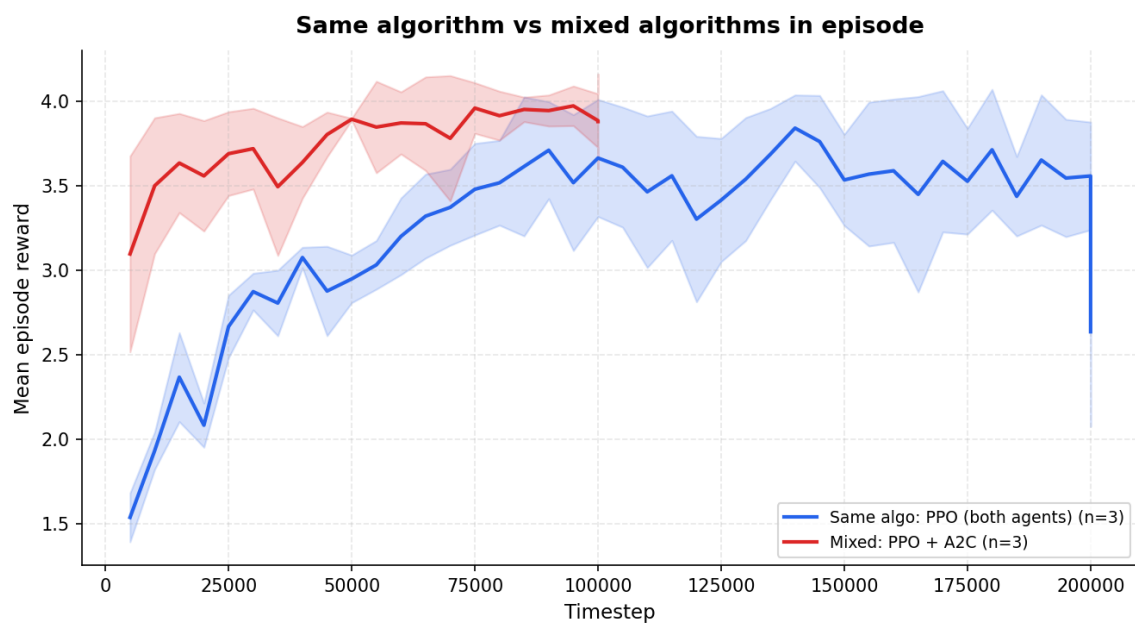
6 Uruchomienie

Kod projektu: `project06/`. Trening: `python -m project06.train_shared`, `python -m project06.train_mixed`

Wykresy: `python -m project06.plot`. Wizualizacja epizodu: `python -m project06.visualize`.



Rysunek 2: Porównanie wariantów hiperparametrów PPO (v0 vs v1).



Rysunek 3: Ten sam algorytm (PPO) vs różne algorytmy w epizodzie (PPO + A2C).